RAW_Image 服务器部署指南（保姆级）

> 目标：把 Windows 本地项目迁移到 Linux 服务器（移动 223.109.239.11 / 电信 180.127.11.169，SSH 端口 **30116**），配置 GPU 环境，跑通 YOLOv8 训练。 前提认知：**Python 代码本身是跨平台的，不用改语言；但依赖包必须按 Linux + GPU 重新装一遍**（Windows 的 conda 环境无法搬过去）。好消息：你的项目代码全部用相对路径（`Path(__file__)...`），基本不需要改路径。

第 0 步：总体流程图

```
本地 VSCode ──Remote-SSH──> 服务器 root 账号
                              │
                    ① 创建个人账户 + sudo
                              │
                    ② 用新账户重连
                              │
                    ③ 代码上传（git clone / scp）
                    ④ 环境重配（Miniconda + GPU PyTorch + 依赖）
                    ⑤ 数据集上传（scp，只传必要的）
                    ⑥ 数据格式转换 + YOLOv8 训练（nohup/tmux）
```

第 1 步：VSCode Remote-SSH 连接 + 创建个人账户

1.1 安装插件

VSCode 扩展市场搜索安装 **Remote - SSH**（微软官方，作者 Microsoft）。

1.2 配置 SSH 连接

按 `Ctrl+Shift+P` → 输入 `Remote-SSH: Open SSH Configuration File` → 选第一个（`C:\Users\98035\.ssh\config`）→ 写入：

```
Host mygpu
    HostName 223.109.239.11
    Port 30116
    User root
```

> 移动线路不通就换成 `HostName 180.127.11.169`。建议把两个都写上（mygpu-mobile / mygpu-telecom），哪条通连哪条。

`Ctrl+Shift+P` → `Remote-SSH: Connect to Host` → 选 `mygpu` → 输入 root 密码。

1.3 创建你的普通账户（以用户名 `zeng` 为例，换成你喜欢的）

```
# 创建用户（-m 建主目录，-s 指定 bash）
useradd -m -s /bin/bash zeng

# 设置密码（输入两次，不会回显）
passwd zeng

# 赋予 sudo 权限（Debian/Ubuntu 系）
usermod -aG sudo zeng
# 如果是 CentOS/Rocky 系用：
# usermod -aG wheel zeng

# 验证
su - zeng -c "sudo whoami"   # 应输出 root
```

1.4 【推荐】配置 SSH 密钥免密登录（一劳永逸）

在你**本地 Windows 的 git-bash 或 PowerShell** 执行：

```
# 本地生成密钥（一路回车）
ssh-keygen -t ed25519 -C "zeng-mygpu"

# 上传公钥到服务器（需要输入一次密码）
ssh-copy-id -p 30116 root@223.109.239.11
```

然后把 1.2 的配置改成：

```
Host mygpu
    HostName 223.109.239.11
    Port 30116
    User zeng
```

之后 VSCode 重连 `mygpu` 即免密登录你的个人账户。

> 安全提醒：服务器 IP、端口、密码属于敏感信息，**不要写进任何会提交到 git 的文件**。

第 2 步：代码上传

方案 A（首选）：服务器上直接 git clone

```
# 在服务器上（你的个人账户）执行
cd ~
git clone https://github.com/Hunter-Stuart/RAW_Image.git
cd RAW_Image && git checkout feature/fix-core-bugs
```

- 你的 `.gitignore` 已经排除了所有数据集（AODRaw、KITTI、权重），**clone 下来的只有代码**，无需担心 97GB 的 zip；
- 如果服务器直连 GitHub 超时（国内服务器常见），依次尝试：

```
# 镜像加速（第三方镜像，仅用于拉取公开代码）
git clone https://ghproxy.com/https://github.com/Hunter-Stuart/RAW_Image.git
# 或下载分支压缩包
wget https://ghproxy.com/https://github.com/Hunter-
Stuart/RAW_Image/archive/refs/heads/feature/fix-core-bugs.zip
unzip feature-fix-core-bugs.zip && mv RAW_Image-feature-fix-core-bugs
RAW_Image
```

方案 B（兜底）：本地 scp 上传

在**本地 git-bash** 执行（Windows 自带 OpenSSH，git-bash 里直接可用）：

```
cd /d/firstwork
# 只传代码目录（binocular_isp），排除一切数据
scp -P 30116 -r RAW_Image/binocular_isp
zeng@223.109.239.11:~/RAW_Image_binocular_isp
```

> 避坑清单（scp 时**不要**传的）：`AODRaw_test.zip`(97GB)、`AODRaw_downsampled_srgb`(4.3G，后面单独传)、`data_scene_flow*`、`SpinnakerSDK*.exe`、任何 `.venv/`、`__pycache__/`。用方案 A 的 git clone 则天然免疫这些问题。
>
> VSCode 拖拽上传：在 Remote-SSH 连接后，资源管理器面板可以直接把本地文件拖进远程目录——适合单文件/小目录，不适合几百 MB 以上的数据。

第 3 步：环境重配（个人账户下）

3.1 摸清服务器情况

```
nvidia-smi          # GPU 型号 + 右上角 CUDA Version（如 12.4）
which conda         # 是否已装 conda（这类 GPU 云服务器常预装）
free -h && df -h    # 内存和磁盘
```

3.2 安装 Miniconda（若 `which conda` 为空）

```
cd ~
# 用清华镜像下载（服务器在国内，快很多）
wget https://mirrors.tuna.tsinghua.edu.cn/anaconda/miniconda/Miniconda3-latest-
Linux-x86_64.sh
```

```
bash Miniconda3-latest-Linux-x86_64.sh   # 一路 yes/回车
source ~/.bashrc                          # 使 conda 生效
```

3.3 创建虚拟环境并安装 GPU 版 PyTorch（关键步骤，注意顺序）

```
conda create -n rawisp python=3.10 -y
conda activate rawisp

# ★ 坑1：必须先装 GPU 版 torch，再装 ultralytics！
#   （pip install ultralytics 会"贴心"地自动装 CPU 版 torch，
#     先装好 GPU 版之后它就不会覆盖）
# ★ 按 nvidia-smi 的 CUDA 版本选 index-url：
#     CUDA 12.x → cu121（或 cu124）
#     CUDA 11.8 → cu118
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121

# 验证 GPU 可用（必须输出 True！）
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"

# 然后装 ultralytics + 项目依赖（国内加速加 -i 清华源）
pip install ultralytics -i https://pypi.tuna.tsinghua.edu.cn/simple
pip install rawpy opencv-python PyYAML tqdm matplotlib pytest -i
https://pypi.tuna.tsinghua.edu.cn/simple
```

> 你的 `requirements.txt` 里没有 torch/ultralytics（本地是手动装的），服务器上按上面的命令手动装即可，或以后把这两行补进 requirements.txt。

3.4 冒烟测试

```
cd ~/RAW_Image/binocular_isp
python tests/test_pipeline.py        # 4/4 通过 = 环境 OK
python -c "from ultralytics import YOLO; YOLO('yolov8n.pt'); print('YOLO OK')"
```

第 4 步：数据集传输（只传训练需要的）

4.1 训练需要的数据（就两样）

数据	大小	用途
`AODRaw_downsampled_srgb/`（7785 张 JPG）	4.3G	训练图像
`AODRaw_annotations/`（COCO 标注 JSON）	21M	转成 YOLO 标签

不需要：97GB 的 `AODRaw_test.zip`、KITTI（`data_scene_flow*`）、Spinnaker 安装包。

4.2 scp 上传（本地 git-bash 执行）

```
cd /d/firstwork/RAW_Image
scp -P 30116 -r AODRaw_downsampled_srgb zeng@223.109.239.11:~/RAW_Image/
scp -P 30116 -r AODRaw_annotations     zeng@223.109.239.11:~/RAW_Image/
```

- 4.3G 大约 10~40 分钟（取决于上行带宽）；scp 断了重跑会覆盖重传，网络不稳时建议装 rsync 续传（服务器：`sudo apt install rsync`，本地 git-bash 也有 rsync 可用：`rsync -avzP -e "ssh -p 30116" ...`）；
- 传完在服务器上核对数量：`ls AODRaw_downsampled_srgb/AODRaw/images_downsampled_srgb/ | wc -l`（应为 7785）。

第 5 步：代码调整（比你想象的少）

你的代码全部用相对路径（`Path(__file__).resolve().parent.parent` 定位项目根），**基本无需修改**。到服务器上做一次体检：

```
cd ~/RAW_Image
# 查有无 Windows 绝对路径/盘符残留（应无输出或只有注释）
grep -rn "D:\\\\" --include="*.py" binocular_isp/ | grep -v "#"
# 查有无反斜杠路径拼接（应无输出）
grep -rn "\\\\\\\\" --include="*.py" binocular_isp/ | grep -v "#"
```

如有命中，把 `D:\xxx\yyy` 改成 `Path(__file__)...` 相对写法或服务器绝对路径 `~/RAW_Image/...`。

第 6 步：YOLOv8 训练（含数据格式转换 + 断线保护）

6.1 转换 COCO 标注 → YOLO 格式（已为你写好脚本）

```
cd ~/RAW_Image
conda activate rawisp
python binocular_isp/scripts/coco_to_yolo.py \
    --ann
AODRaw_annotations/AODRaw/annotations/train_annotations_downsample_scale3_bbox_min
_size32.json \
    --img-dir AODRaw_downsampled_srgb/AODRaw/images_downsampled_srgb \
    --out ~/datasets/aodraw
```

产物：

```
~/datasets/aodraw/
├── images/train -> (符号链接到 4.3G 图像目录，不复制)
├── labels/train/*.txt     # 每行: cls cx cy w h（归一化）
└── data.yaml              # 数据集配置（path/train/names）
```

脚本只保留与 COCO 同名的约 33 个类别（AODRaw 63 类里与 YOLO 预训练类别能对应的部分）。

6.2 启动训练（三种防断线方式）

方式 1：nohup（最简单）

```
cd ~/RAW_Image/binocular_isp
nohup yolo train detect \
    model=yolov8n.pt \
    data=~/datasets/aodraw/data.yaml \
    epochs=50 imgsz=640 batch=16 device=0 \
    project=~/train_results name=aodraw_v1 \
    > ~/train.log 2>&1 &

tail -f ~/train.log          # 实时看训练日志
```

方式 2：tmux（推荐，可随时"回到"训练现场）

```
tmux new -s train            # 新建会话
# 在会话里运行训练命令（去掉 nohup 和 &）
yolo train detect model=yolov8n.pt data=~/datasets/aodraw/data.yaml \
    epochs=50 imgsz=640 batch=16 device=0 project=~/train_results name=aodraw_v1

# 按 Ctrl+B 然后按 D = 脱离会话（训练继续跑），断开 SSH 也不影响
tmux attach -t train         # 重新进入会话
tmux ls                      # 查看所有会话
```

> 没有 tmux 时先装：`sudo apt install tmux`（或 `conda install -c conda-forge tmux`）。screen 用法类似（`screen -S train` / `Ctrl+A D` / `screen -r`），二选一即可。

方式 3：VSCode 终端 + 上述任一种——VSCode 的 Remote-SSH 终端同样会因网络断开而杀死前台进程，所以**任何长时间训练都必须套 nohup 或 tmux**。

6.3 训练监控与结果回传

```
watch -n 5 nvidia-smi          # 每5秒刷新GPU占用
tail -f ~/train.log            # 日志
# 训练完产物在 ~/train_results/aodraw_v1/weights/best.pt 和 results.png

# 结果回传本地（本地 git-bash 执行）
scp -P 30116 zeng@223.109.239.11:~/train_results/aodraw_v1/results.png
/d/firstwork/
scp -P 30116 zeng@223.109.239.11:~/train_results/aodraw_v1/weights/best.pt
/d/firstwork/RAW_Image/binocular_isp/data/
```

6.4 参数建议（按你的服务器情况调整）

参数	建议	说明
model	yolov8n.pt	先小模型跑通全流程，再换 yolov8s/m
epochs	50	7785 张图、单 GPU 约 1~2 小时
imgsz	640	常规起步值
batch	16	显存 8G 起步；OOM 就降到 8
device	0	多卡可写 0,1
workers	8	加速数据加载

常见坑速查表

坑	解法
`torch.cuda.is_available()` 输出 False	① 确认 `nvidia-smi` 有输出；② 确认 torch 是 cu 版（`pip list
ultralytics 自动装了 CPU torch	按第 3.3 步顺序：先 torch 后 ultralytics
git clone GitHub 超时	ghproxy 镜像 / 压缩包方式（第 2 步方案 A）
scp 传大文件断线	rsync -avzP 续传
SSH 断开训练就停了	必须 nohup 或 tmux 包裹
显存不足 (CUDA out of memory)	batch 减半、imgsz 降到 512
pip 下载慢	加 `-i https://pypi.tuna.tsinghua.edu.cn/simple`
端口 30116 连不上移动线路	换电信 IP `180.127.11.169`
磁盘不够	`df -h` 看数据盘挂载点（GPU 云常把大磁盘挂 `/root/workspace` 或 `/data`），数据集放大的分区

配套脚本 `binocular_isp/scripts/coco_to_yolo.py` 已随代码仓库上传。